CrowdArb

Aris Rault

July 2026

CrowdArb is a cross-market probability calibration platform for binary prediction-market contracts. It pairs each Polymarket contract with a professional-market counterpart — CME Fed Funds Futures for rate decisions, Black-Scholes on Deribit implied volatility for crypto price levels — and blends the two probability estimates into a single fair value. That fair value feeds an Avellaneda-Stoikov inventory-aware market-maker and a Bayesian model-averaging calibrator that learns online which source to trust more. The core thesis is that prediction-market crowds and professional markets frequently disagree on the same binary event, and that disagreement is exploitable with a principled quoting and calibration framework. A live Streamlit scanner discovers priceable Polymarket contracts automatically and surfaces the largest crowd-vs-professional gaps across all tracked markets simultaneously.

Live system: https://crowdarb.streamlit.app **Repository:** https://github.com/Arisrlt46/crowdarb

1 Results

Tested on 200-step synthetic price series with fixed random seeds.

Layer	Metric	Baseline	CrowdArb	Improvement
2 — Avellaneda-Stoikov (Fed)	P&L variance	0.6621	0.4145	−37.4%
2 — Avellaneda-Stoikov (Bitcoin)	P&L variance	4.4339	1.7658	−60.2%
3 — Bayesian blending (Fed)	Brier score	0.2186	0.2095	−4.1%
3 — Bayesian blending (Bitcoin)	Brier score	0.0112	0.0101	−9.6%

Inventory-aware quoting reduces P&L variance by 37%–60% across markets. Bayesian model averaging improves calibration by up to 9.6% relative to using Polymarket alone.

2 Architecture

Layer	Files	Role
0	`layer0_bitcoin.py`, `layer0_ethereum.py`, `layer0_compare.py`, `layer0_fedwatch.py`, `layer0_bs.py`, `layer0_markets.py`, `layer0_classifier.py`	Data ingestion and contract discovery — live prices, implied volatilities, and futures-derived probabilities
1	`layer1_belief.py`, `layer1_backtest.py`	Beta-Bernoulli belief updater; naive symmetric market-maker; backtest harness
2	`layer2_avellaneda.py`	Avellaneda-Stoikov reservation price and optimal spread; inventory-aware quoting
3	`layer3_calibration.py`	Bayesian model averaging with online Hedge weight learning
4	`layer4_llm_signal.py`	LLM signal layer — news headline in, likelihood ratio out, belief updated
Interface	`market_pair.py`	`MarketPair` abstract base class; `FedRateMarket`, `BitcoinMarket`, `EthereumMarket` implementations; `discover_markets()`
Dashboard	`dashboard_scanner.py`	Streamlit multi-market scanner with auto-discovery, A-S live quotes, and headline scorer

3 Live markets

Fed rate decisions. `FedRateMarket` pairs the Polymarket "no Fed rate cuts in 2026" contract against a CME FedWatch probability derived from 30-day Fed Funds Futures (ZQ contracts). The formula back-solves the post-meeting implied rate from the monthly weighted average; when the meeting falls in the last few days of the month (< 5 remaining days), it switches to the next month's ZQ contract to avoid arithmetic amplification from a near-zero denominator — matching CME's own published methodology.

Bitcoin price levels. `BitcoinMarket` finds the highest-volume Polymarket "reach $X" contract with at least 30 days to expiry and a Black-Scholes probability in [2%, 98%] — filtering out near-certain outcomes where crowd-vs-professional comparison is uninformative. The professional probability uses Black-Scholes with Deribit's 30-day DVOL index as implied volatility, falling back to 90-day historical vol if Deribit is unavailable.

Ethereum price levels. `EthereumMarket` mirrors the Bitcoin implementation using ETH-USD spot and Deribit ETH DVOL.

4 Auto-discovery

Rather than hardcoding market names, `discover_markets()` in `market_pair.py` scans the full live Polymarket catalogue on each refresh. `classify_market()` in `layer0_classifier.py` tags each contract as `crypto_level`, `rate_decision`, `equity_level`, or `unsupported` using regex patterns with word-boundary guards. Contracts pass three filters before a `MarketPair` is instantiated: minimum volume ($50k), minimum days to expiry (30), and a Black-Scholes uncertainty window (professional probability between 2% and 98%). Within each (type, underlying) group the highest-volume contract wins; the scanner table is populated entirely from what discovery returns. Adding support for a new asset requires only extending the classifier patterns — no new loader functions or dashboard code.

5 LLM news-signal layer

Layer 4 (`layer4_llm_signal.py`) takes a plain-text news headline and returns a structured likelihood ratio — P(headline | event occurs) / P(headline | event does not occur) — using Claude. The ratio is applied as a Bayesian update on the current Beta-Bernoulli belief, shifting the fair value and therefore the A-S reservation price and spread. In the Streamlit dashboard the scorer is scoped per market so results persist across market switches.

6 Run instructions

Setup:

```
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # add ANTHROPIC_API_KEY (required for Layer 4 only)
```

Per-market backtests:

```
python3 layer2_avellaneda.py   # Fed: A-S vs naive, saves layer2_backtest.csv + chart
python3 layer2_bitcoin.py     # Bitcoin: A-S vs naive, saves layer2_bitcoin_backtest.csv
python3 layer3_calibration.py # Fed: Bayesian blending, saves layer3_calibration.csv + chart
python3 layer3_bitcoin.py      # Bitcoin: Bayesian blending
```

Full pipeline on all three live markets:

```
python3 market_pair.py         # runs Layers 1-3 on Fed, Bitcoin, Ethereum end-to-end
```

Auto-discovery demo:

```
python3 -c "from market_pair import discover_markets; discover_markets()"
```

Streamlit scanner dashboard:

```
streamlit run dashboard_scanner.py
```

The dashboard auto-discovers markets on first load (≈15 s cold start), caches results for 5 minutes, and displays a ranked scanner table sorted by the absolute Polymarket-vs-professional gap. The Refresh button clears the discovery cache. The detail panel shows live A-S quotes and a headline scorer for the selected market.

All backtests use fixed random seeds for reproducibility (`layer2_avellaneda.py`: seed=42, p0=0.72; `layer3_calibration.py`: seed=7, p_true=0.30).

7 Requirements

Python 3.9+. Dependencies in `requirements.txt`. An `ANTHROPIC_API_KEY` is required only for Layer 4 (LLM headline scorer).

8 License

MIT